

SecFlowOps: A Controlled DevSecOps Pipeline Artifact for Scanner, Remediation, and Policy-Gate Evaluation

Anonymous for review

Abstract—This manuscript reports SecFlowOps, an executable artifact for evaluating DevSecOps pipeline configurations under scanner, remediation, and policy-gate workloads. The artifact executes Trivy, Semgrep, Gitleaks, npm audit, OWASP ZAP, and OPA; normalizes findings; applies deterministic and npm-based remediations; rescans patched workspaces; and records security and performance evidence. The evaluation comprises a 432-run full controlled protocol, a 90-run controlled size study, a 75-run open-source campaign with static adjudication, a NodeGoat npm-remediation study, an injected external recall study, and a targeted ZAP DAST probe. SecFlowOps reaches recall 1.0 on injected external ground truth and removes all injected target findings after remediation. On NodeGoat, npm-only remediation reduces findings from 78 to 32, while SecFlowOps reduces them to 31 by also removing the repository secret; residual critical findings remain correctly blocked by policy. The results support SecFlowOps as a reproducible evaluation artifact, not as a general proof that arbitrary projects can be automatically repaired.

I. INTRODUCTION

DevSecOps pipelines often combine scanners, dependency updates, local patches, and deployment gates, but empirical claims about these combinations are difficult to audit. A scanner-only pipeline can find issues without changing delivery behavior. A remediation pipeline can edit code without proving that the residual state satisfies an explicit policy. A policy gate can block delivery without showing whether remediation actually reduced risk.

SecFlowOps is an executable evaluation artifact for this gap. It compares configurations ranging from scanning-only execution to scanner, remediation, rescan, and OPA-based policy enforcement. The contribution is not a new vulnerability scanner. It is a reproducible protocol for measuring how scanner findings move through remediation and policy gates, including cases where the correct outcome is refusal rather than delivery.

This revision addresses four reviewer-facing weaknesses. First, it adds result tables that connect campaign design, residual findings, and performance. Second, it separates injected-ground-truth recall from conservative scanner-output precision and static adjudication. Third, it adds baselines, including npm-audit-only remediation on NodeGoat. Fourth, it expands the paper from a smoke-run description into a fuller artifact evaluation with limitations stated at the level supported by the experiments.

II. BACKGROUND AND RELATED WORK

Secure-development frameworks such as NIST SSDF and OWASP SAMM emphasize vulnerability management, feedback loops, and measurable secure-development activities [1], [2]. These frameworks motivate automation but do not prescribe a concrete experimental method for comparing scanner-only, remediation, and policy-gated DevSecOps pipelines.

SecFlowOps builds on existing tools rather than replacing them. Trivy provides vulnerability, secret, and misconfiguration scanning over filesystems and language ecosystems [3]. Semgrep supplies rule-driven SAST for source code [4]. Gitleaks detects repository secrets [5]. npm audit supplies advisory-driven JavaScript dependency remediation [6]. OPA provides policy-as-code enforcement through Rego and structured input decisions [7].

The artifact contribution is the integration and measurement layer: normalized finding records, explicit ground-truth linkage where available, deterministic remediation traces, post-remediation rescans, and policy decisions over residual findings. This places SecFlowOps closer to a reproducible benchmark harness than to a standalone AppSec product.

III. METHOD

The pipeline has five stages. First, build or test commands are executed in a copied workspace. For heterogeneous external repositories, the build step can be explicitly skipped so that missing project-specific dependencies do not confound security-pipeline measurement. Second, scanners collect SAST, SCA, secret, Docker, and Kubernetes evidence. Third, findings are normalized into JSONL records with stable fingerprints and optional ground-truth links. Fourth, remediation rules patch only the copied workspace. Fifth, a second scanner pass measures residual findings before OPA evaluates the delivery policy.

Remediation is deliberately bounded. Deterministic rules update known vulnerable Python pins, remove documented fake secrets, escape reflected HTML output, parameterize a SQL query, and harden Docker or Kubernetes settings. For NodeGoat, the npm remediator runs `npm audit fix --package-lock-only --omit=dev` and records the npm return code and lockfile change. A non-zero npm return code is retained as evidence of residual advisories, not hidden as a tool failure.

OPA evaluates residual findings using a policy with zero tolerance for critical findings, a high-severity threshold of

three, a CVSS ceiling of 9.0, and blocking on residual secrets. The policy counts residual findings as a list rather than a set so duplicate scanner findings are not silently deduplicated.

IV. EXPERIMENTAL DESIGN

Table I summarizes the campaigns. The controlled campaign evaluates six configurations: build only, non-blocking scanning, automatic scanning, policy only, agents only, and the combined SecFlowOps configuration. Five generated repositories inject SAST, SCA, secret, Docker, and Kubernetes findings with explicit ground-truth files. Each controlled repository is evaluated three times, yielding 90 runs.

The full protocol campaign materializes the preregistered local design: three repository families, eight scenarios, six configurations, and three repetitions, yielding 432 runs. The eight scenarios are clean, vulnerable dependency, fake secret, reflected XSS, SQL injection, Docker misconfiguration, Kubernetes misconfiguration, and multi-layer. This campaign was completed before ZAP was enabled in the runner; ZAP is therefore evaluated in a separate targeted DAST probe.

The external campaign evaluates five configurations, non-blocking scanning, automatic scanning, policy only, agents only, and the combined SecFlowOps configuration, on five open-source repositories: Requests, Flask, Express, OWASP NodeGoat, and DVNA. Each external repository is evaluated three times, yielding 75 runs. The external campaign uses `--build-mode skip` to avoid confounding security-pipeline behavior with project-specific dependency installation and test setup.

Two focused campaigns address external validity gaps. A NodeGoat npm study repeats all five configurations three times on OWASP NodeGoat and enables `npm audit fix --package-lock-only --omit=dev` in the remediation workspace. A separate npm-audit-only baseline repeats the same package-manager remediation three times without SecFlowOps secret remediation or OPA gating. An injected external recall study copies Requests, Flask, and Express, injects six controlled findings under `secflowops_injected/`, and evaluates three configurations over three repetitions, yielding 27 runs with complete injected ground truth.

V. SECURITY RESULTS

The generated tables are stored in `SecFlowOps/tables`. Table II summarizes the main security outcomes across the controlled, external, NodeGoat, and injected-external studies.

In the expanded controlled campaign, five generated repositories were evaluated across six configurations and three repetitions, for 90 runs. The policy-only configuration was blocked in all 15 runs, while the combined SecFlowOps configuration passed in all 15 runs after local remediation and OPA evaluation. For SecFlowOps runs, residual critical findings and residual secrets were reduced to zero. Two high-severity Kubernetes hardening findings remained in each run and were permitted by the configured policy threshold.

The full protocol campaign executed 432/432 planned local runs. Each configuration has 72 runs. Pipeline success was

72/72 for C0, C1, C2, C4, and C5, and 45/72 for C3 because policy-only correctly denied vulnerable raw findings. Mean residual findings decreased from 28.0 in scan-only configurations to 16.0 in C4 and C5. Mean recall was 0.795 in the 432-run campaign because DAST ground-truth rows were included before ZAP was enabled in that campaign. A subsequent targeted ZAP probe produced real ZAP JSON output and matched the reflected-XSS DAST ground truth with severity high.

In the external campaign, 75 runs were executed with no tool failures. Non-blocking scanning, automatic scanning, and agents-only execution passed in 15/15 runs. Policy-only and SecFlowOps each passed in 12/15 runs; the failed runs were policy denials on OWASP NodeGoat. The external campaign produced 82 unique findings and a first-pass static adjudication table. Under this adjudication, SecFlowOps resolved 13/13 Flask example dependency findings, 2/2 DVNA Dockerfile hardening findings, and the NodeGoat repository-stored private key.

Table III isolates the hardest external repository. The added npm-only baseline shows that `npm audit fix --package-lock-only --omit=dev` resolves many NodeGoat dependency advisories but leaves the repository secret untouched. SecFlowOps reaches a slightly lower residual count because it also removes the committed private key. Residual findings still include 3 critical and 12 high findings, so SecFlowOps remained blocked by policy in all three combined runs.

The injected external recall study completed 27/27 runs successfully. Table IV distinguishes detection from remediation. Recall against injected ground truth was 1.0 for all evaluated configurations. For SecFlowOps, all nine runs ended with zero residual critical findings, zero residual secrets, and zero residual injected ground-truth findings.

A. Precision Interpretation

We report three distinct notions that should not be merged. First, controlled and injected-external recall are measured against known injected ground truth. Second, the conservative precision field in the generated summary tables treats all non-ground-truth scanner findings as unknown or false-positive candidates; for the injected external study this yields a mean precision of 0.088 because real repository hardening and dependency findings coexist with the injected findings. This number is therefore a lower-bound bookkeeping measure, not a claim that contemporary scanners are mostly wrong. Third, the external adjudication table reviewed 82 unique naturally occurring findings by static repository evidence; within that reviewed subset, adjudicated precision was 1.0 for runs with adjudicated findings. This is not an independent exploitability audit and is not used to claim complete recall over naturally occurring external vulnerabilities.

VI. PERFORMANCE RESULTS

The expanded campaign includes a performance study using the same 90 runs. Table V summarizes the main end-to-end

TABLE I
EVALUATION CAMPAIGNS AND MAIN PURPOSE.

Campaign	Corpus	Runs	Reps	Purpose
Controlled	five generated API variants	90	3	End-to-end behavior with injected ground truth and controlled size variation.
Full protocol	24 generated scenario repositories	432	3	Preregistered local design: three families, eight scenarios, six configurations, and three repetitions.
External OSS	Requests, Flask, Express, NodeGoat, DVNA	75	3	Scanner, remediation, adjudication, policy, and performance behavior on real repositories.
NodeGoat npm	OWASP NodeGoat	15+3	3	Scanner-only, npm-audit-only, agents-only, and SecFlowOps behavior on a difficult npm dependency graph.
Injected external	Requests, Flask, Express copies with injected findings	27	3	Recall and residual target findings where complete injected ground truth exists in real repository contexts.
ZAP DAST probe	reflected-XSS generated repository	2	1	Validate OWASP ZAP execution, JSON normalization, and DAST ground-truth matching.

TABLE II
MAIN SECURITY OUTCOMES. COUNTS ARE MEANS WHEN SEVERAL REPOSITORIES OR REPETITIONS ARE AGGREGATED.

Study and configuration	Runs	Success	Initial	Residual	Main interpretation
Controlled C1	15	15	64.0	64.0	Scanner-only baseline; injected-ground-truth recall was 1.0.
Controlled C5	15	15	64.0	16.0	Critical findings and secrets were reduced to zero; two high Kubernetes hardening findings remained per run.
External C1	15	15	18.6	18.6	Natural OSS scan without remediation.
External C5	15	12	18.6	15.4	Flask and DVNA findings were remediated; NodeGoat was denied by policy.
NodeGoat C5	3	0	78.0	31.0	npm remediation and secret removal were partial; policy denial remained correct.
Injected external C5	9	9	68.3	16.0	Injected-ground-truth recall was 1.0 and residual injected findings were zero.

TABLE III
NODEGOAT NPM REMEDIATION AND POLICY OUTCOME.

Configuration	Total	Crit.	High	Secrets	Policy
C1 scanner only	78	10	40	1	N/A
B1 npm audit only	32	3	13	1	N/A
C4 agents only	31	3	12	0	N/A
C5 SecFlowOps	31	3	12	0	Deny

TABLE IV
INJECTED-EXTERNAL RECALL AND REMEDIATION.

Configuration	Runs	Recall	Resid. GT	Outcome
C1 scanner only	9	1.0	N/A	Detection only
C4 agents only	9	1.0	0	Injected findings re-mediated
C5 SecFlowOps	9	1.0	0	Remediated and policy-passed

timings. Mean pipeline time was 0.246 seconds for build-only execution, 9.181 seconds for non-blocking scanning, 9.345 seconds for policy-only execution, 17.759 seconds for agents-only execution, and 18.122 seconds for the full SecFlowOps

TABLE V
MEAN END-TO-END RUNTIME BY STUDY AND CONFIGURATION.

Study	C1 scan	C5 SecFlowOps	Overhead
Controlled	9.181 s	18.122 s	8.941 s
External OSS	14.671 s	28.637 s	13.966 s
Injected external	18.155 s	35.519 s	17.364 s
NodeGoat npm	19.620 s	96.302 s	76.682 s
Full protocol	22.061 s	43.749 s	21.688 s

configuration.

For SecFlowOps, the mean component times were 8.819 seconds for the initial scan, 8.553 seconds for the post-remediation scan, 0.288 seconds for local remediation, and 0.121 seconds for the OPA policy gate. Thus, scanner execution dominates runtime; policy evaluation is not the primary performance bottleneck in this controlled setting.

The paired mean overhead of SecFlowOps over non-blocking scanning was 8.941 seconds. This overhead is mainly attributable to remediation validation and the second scanner pass.

In the external campaign, mean pipeline time was 14.671

TABLE VI
NODEGOAT NPM PERFORMANCE BASELINE.

Configuration	Pipeline	npm/remediation
B1 npm audit only	104.518 s	51.078 s
C5 SecFlowOps	96.302 s	44.778 s

seconds for non-blocking scanning, 14.133 seconds for automatic scanning, 14.926 seconds for policy-only execution, 27.924 seconds for agents-only execution, and 28.637 seconds for SecFlowOps. For SecFlowOps, mean component times were 14.304 seconds for the initial scanner pass, 13.663 seconds for the post-remediation scanner pass, 0.401 seconds for remediation, and 0.097 seconds for the OPA policy gate. The paired mean overhead of SecFlowOps over non-blocking scanning was 13.966 seconds, again dominated by the second scanner pass.

Table VI compares the NodeGoat npm-only baseline with SecFlowOps. The npm-only baseline took 104.518 seconds on average, including 51.078 seconds in `npm audit fix`. SecFlowOps took 96.302 seconds on average, including 44.778 seconds of remediation. These values should be interpreted as local artifact timings rather than general npm performance constants because package-manager runtime depends on cache state, lockfile structure, and advisory-database state.

In the injected external recall study, mean pipeline time was 18.155 seconds for non-blocking scanning and 35.519 seconds for SecFlowOps. These results show that npm remediation is substantially more expensive than deterministic text patches, and that scanner re-execution remains the dominant overhead across all studies.

In the 432-run full protocol campaign, mean pipeline time was 22.061 seconds for C1 and 43.749 seconds for C5; medians were 20.691 seconds and 41.409 seconds, respectively. One C2 Semgrep run lasted 3944.596 seconds and is retained in the raw data; this outlier explains why C2 mean runtime is much larger than its median. In the ZAP probe, the ZAP-enabled scanner pass took 25.845 seconds for C1 and 20.116 seconds for C3, including ZAP quick scan execution against a local dynamic-port target.

VII. REPRODUCIBILITY AND ARTIFACT SCOPE

The artifact includes executable scripts for the reported studies: `run_expanded_study.ps1`, `run_full_protocol_study.ps1`, `run_external_study.ps1`, `run_nodegoat_npm_study.ps1`, and `run_injected_external_study.ps1`. It also includes a separate npm-only baseline script, `run_npm_audit_only_baseline.ps1`. The scripts generate raw logs, normalized findings, processed run metrics, summary tables, and performance tables under `SecFlowOps/data`, `SecFlowOps/tables`, and `SecFlowOps/figures`.

The checked local toolchain uses Python 3.13, Docker, Java 24.0.1, Trivy 0.71.2, Semgrep 1.169.0, OPA

1.18.2, Gitleaks 8.30.1, and OWASP ZAP 2.17.0. OPA, Gitleaks, and ZAP are bundled in the artifact, while Trivy, Semgrep, Docker, npm, and Java are expected from the host environment. Tool status is recorded in `SecFlowOps/artifact/tool_status.json`; file checksums are recorded in `SecFlowOps/artifact/checksums.sha256`.

Remote workflow evidence is recorded in `github_actions_evidence.md`. The GitHub validation used a private repository, executed the remote CI workflow on `main`, created a real remediation pull request from a user-pushed branch, and observed passing `pull_request` CI on that PR. The PR was left open as an auditable remediation proposal rather than merged.

Reproducibility is claimed at the artifact-workflow level: a reviewer can rerun the scripts to regenerate metrics using the same source state and installed tools. We do not claim bit-identical future vulnerability counts because Trivy, npm, and advisory databases are time-sensitive. We also do not claim independent replication: no third-party rerun has been performed as part of this manuscript.

VIII. THREATS TO VALIDITY

The controlled campaign uses generated repositories and deterministic patches. The ground truth is injected by construction and does not represent natural project diversity.

The external campaign improves external validity by using five real open-source repositories and a first-pass static adjudication of 82 findings. This adjudication verifies repository evidence but is not an independent audit of every CVE status and does not validate application-level exploitability. Recall over naturally occurring external vulnerabilities is not claimed because no complete independent ground truth exists for the external repositories.

The injected external campaign provides complete ground truth only for the injected findings, not for every vulnerability that may naturally exist in the base repositories. External campaigns use build mode `skip`: they measure security-pipeline behavior, but they do not prove that remediated third-party applications still pass their full project-specific test suites. The NodeGoat npm remediation uses non-breaking `npm audit fix`; remaining findings require breaking upgrades, replacement of unmaintained packages, or manual redesign.

The toolchain is time-sensitive because scanner databases, npm advisory metadata, and package-manager resolution behavior change. Tool versions and checksums are recorded in the artifact, but future runs may produce different vulnerability counts as advisory databases evolve.

The controlled precision estimate treats scanner findings not linked to injected ground truth as unknown or false-positive candidates. The external adjudication improves this point for the reviewed findings, but it remains a static first-pass review rather than a blinded multi-reviewer audit. Stronger claims about external false-positive rates require independent reviewers and a broader corpus.

Remote GitHub Actions execution and real pull-request creation were performed on a private repository, with evidence recorded in `github_actions_evidence.md`. The remote validation covers artifact compilation, unit tests, OPA policy tests, workflow syntax validation, a workflow-created remediation branch, and a user-pushed remediation pull request with passing pull-request CI. This evidence resolves the previous local-only CI/PR limitation, but it remains a functional validation rather than a measurement of human review quality, merge acceptance, review latency, branch protection policy, or production deployment after merge.

IX. CONCLUSION

SecFlowOps executes real scanner, remediation, policy, metric, adjudication, DAST, and performance workflows. The full controlled protocol executes 432/432 planned local runs across three repository families, eight scenarios, six configurations, and three repetitions. The earlier controlled size study, the 75-run open-source campaign, the NodeGoat npm-remediation study, and the injected external recall study provide complementary checks on size effects, external repositories, package-manager remediation, and injected ground-truth matching. The added ZAP probe validates executable OWASP ZAP scanning, JSON normalization, and reflected-XSS ground-truth matching under the Java 24 runtime available in the artifact environment.

The resulting contribution is an evaluable DevSecOps artifact and measurement protocol, not a production-ready autonomous remediation system. The evidence now supports claims about local scanner execution, controlled and injected recall, partial external remediation, policy gating, ZAP-enabled DAST execution, measured overhead, remote GitHub Actions execution, and real pull-request creation with passing pull-request CI. It does not support complete recall over naturally occurring OSS vulnerabilities, exploitability of every scanner finding, compatibility preservation for all remediated applications, human acceptance of remediation proposals, or production merge efficacy.

REFERENCES

- [1] M. Souppaya, K. Scarfone, and D. Dodson, "Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-218, 2022. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-218>
- [2] OWASP Foundation, "OWASP SAMM: Software assurance maturity model," 2026, accessed 13 July 2026. [Online]. Available: <https://owasp.org/www-project-samm/>
- [3] Aqua Security, "Trivy documentation," 2026, accessed 13 July 2026. [Online]. Available: <https://trivy.dev/docs/latest/guide/>
- [4] Semgrep, "Semgrep documentation," 2026, accessed 13 July 2026. [Online]. Available: <https://docs.semgrep.dev/>
- [5] Gitleaks Project, "Gitleaks," 2026, accessed 13 July 2026. [Online]. Available: <https://github.com/gitleaks/gitleaks>
- [6] npm, Inc., "npm-audit cli documentation," 2026, accessed 13 July 2026. [Online]. Available: <https://docs.npmjs.com/cli/v11/commands/npm-audit/>
- [7] Open Policy Agent, "Open policy agent documentation," 2026, accessed 13 July 2026. [Online]. Available: <https://www.openpolicyagent.org/docs>